

PAT1:-

Single File Programming Question
Problem Statement

Raj is working in a mobile phone repair shop, and he needs to calculate the total time required to replace components in a mobile phone. He has three types of components to replace: screens, batteries, and cameras. Each component has a standard time required for replacement.

Raj needs to calculate the time spent on each type of component and the total time required for the repair.

Replacing a screen takes 15 minutes per unit.
Replacing a battery takes 5 minutes per unit.
Replacing a camera takes 15 minutes per unit.

Write a program that calculates the total repair time based on the number of screens, batteries, and cameras to be replaced.

Input format :

The first line contains an integer C1, representing the number of screens to be replaced.

The second line contains an integer C2, representing the number of batteries to be replaced.

The third line contains an integer C3, representing the number of cameras to be replaced.

Output format :

The first line of output prints "Time for Screen Replacement: X minutes" where X is the calculated time for screen replacement.

The second line prints "Time for Battery Replacement: Y minutes" where Y is the calculated time for battery replacement.

The third line prints "Time for Camera Replacement: Z minutes" where Z is the calculated time for camera replacement.

The fourth line prints "Total Repair Time: W minutes" where W is the total calculated repair time.

Refer to the sample output for the exact text and format.

Code constraints :

In this scenario, the test cases fall under the following constraints:

$1 \leq C1, C2, C3 \leq 500$

Sample test cases :

Input 1 :

5

1

3

Output 1 :

Time for Screen Replacement: 75 minutes

Time for Battery Replacement: 5 minutes

Time for Camera Replacement: 45 minutes

Total Repair Time: 125 minutes

Input 2 :

25

10

7

Output 2 :

Time for Screen Replacement: 375 minutes

Time for Battery Replacement: 50 minutes
Time for Camera Replacement: 105 minutes
Total Repair Time: 530 minutes

Question No: 2

Single File Programming Question
Problem Statement

Sarah is a property manager who needs to calculate the monthly rent for her tenants based on the type of property and the duration of their stay. The rent calculation includes discounts for long-term stays and additional fees for property maintenance. The additional fee is calculated as 2% of the base monthly rent per year of stay.

The properties and their base monthly rents are as follows:

Apartment: 1000 dollars/month

House: 1500 dollars/month

Villa: 2000 dollars/month

Discounts based on the duration of stay are applied as follows:

1-3 years: 5%

4-6 years: 10%

7+ years: 15%

Given the type of property and the duration of stay, write a program to calculate the final monthly rent for a tenant, including additional maintenance fees.

Note: The discount, maintenance fee, and final monthly rent are calculated using double.

Input format :

The first line contains an integer P, representing the property type (1 for Apartment, 2 for House, 3 for Villa).

The second line contains an integer Y, representing the duration of stay in years.

Output format :

The output displays an integer representing the final monthly rent for the tenant.

Refer to the sample output for formatting specifications.

Code constraints :

P can be 1, 2, or 3

$1 \leq Y \leq 15$

Sample test cases :

Input 1 :

1

2

Output 1 :

990

Input 2 :

3

8

Output 2 :

2020

PAT2:-

Question No: 1

Single File Programming Question

Problem Statement

Sandeep is organizing a neighborhood bicycle race and he wants to rearrange the starting positions of the participants.

Write a program that takes an array of participant numbers and an integer M and rotates the array to the right by M positions to determine the new starting order for the race.

Input format :

The first line of input consists of an integer N, representing the number of participants.

The second line consists of N space-separated integers, representing the participant numbers.

The third line consists of an integer M, representing the number of positions to be rotated.

Output format :

The output prints the rotated array with M positions to the right, separated by a space.

Refer to the sample output for formatting specifications.

Code constraints :

$1 \leq N \leq 10$

$1 \leq \text{elements} \leq 100$

$1 \leq M < N$

Sample test cases :

Input 1 :

5
47 56 95 24 38

3
Output 1 :
95 24 38 47 56

Input 2 :

8
1 2 3 4 5 6 7 8

6
Output 2 :
3 4 5 6 7 8 1 2

Question No: 2

Single File Programming Question

Problem Statement

In a financial auditing system, each transaction ID is analyzed to identify significant amounts based on certain criteria. Specifically, the system needs to calculate the sum of all odd digits in the transaction ID that is greater than 5. This helps in detecting high-value transactions with specific characteristics.

Given a transaction ID, write a program to compute the sum of all odd digits in the ID that are greater than 5.

Input format :

The input consists of a single integer n, representing the transaction ID.

Output format :

The output displays a single integer representing the sum of odd digits greater

than 5 in the transaction ID.

Refer to the sample output for formatting specifications.

Code constraints :

$1 \leq n \leq 10^6$

Sample test cases :

Input 1 :

12345

Output 1 :

0

Input 2 :

8972

Output 2 :

16

Input 3 :

578

Output 3 :

7

PAT3:-

Question No: 1

Single File Programming Question

Problem Statement

Tejas is working on a problem where he needs to analyze the digits of a given number. Specifically, he wants to find the difference between the sum of the even digits and the sum of the odd digits in a number.

To accomplish this, Tejas decides to write a Java program. Tejas creates a class named DigitDifference in his program and uses it to perform the calculation.

Note:

If there are no even digits in the given input number, the sum of the even digits is assumed to be zero.

If there are no odd digits in the given input number, the sum of the odd digits is also assumed to be zero.

The final output will display the difference between the sum of even digits and the sum of odd digits.

Input format :

The first line of input contains an integer, T representing the number of test cases.

Each of the next T lines contains a positive integer N, representing the number whose digits will be analyzed.

Output format :

For each test case, the output prints the difference between the sum of the even digits and the sum of the odd digits of the number on a new line.

Refer to the sample output for the formatting specifications.

Code constraints :

$1 \leq T \leq 10$

$1 \leq N \leq 1000$

Sample test cases :

Input 1 :

6
882
217
840
991
278
636

Output 1 :

18
-6
12
-19
3
9

Question No: 2

Single File Programming Question
Problem Statement

John is the owner of a coffee shop and wants to track the sales his various coffee blends. He has asked you to develop a Java program to manage this. You are required to create a class named CoffeeBlend that will store details about each coffee blend, including its name, price, and quantity sold. Create an array of objects to represent multiple coffee blends.

The class should use a parameterized constructor to initialize these attributes: name, price, and quantity.

A static method calculateDiscount(int quantitySold) should calculate the discount based on the following conditions:

If more than 9 units are sold, the discount is 20% (0.20).

If more than 3 units are sold, the discount is 10% (0.10).

Otherwise, there is no discount (0.0).

Formula used:

Total Sales Value: price * quantity sold

Discounted Sales Value: totalSalesValue * (1 - discount).

Input format :

The first line contains an integer n, the number of coffee blends.

For each coffee blend following input is given:

A string represents the name of the coffee blend.

A double represents the price per unit of the coffee blend.

An integer represents the quantity of the coffee blend sold.

Output format :

The output displays the coffee blend and discounted sales value separated by space.

Refer to the sample output for formatting specifications.

Code constraints :

$1 \leq n \leq 10$

$1.0 \leq \text{price} \leq 1000.0$

$1 \leq \text{quantity} \leq 500$

$1 \leq \text{length of coffee blend name} \leq 100$

The fixed cost per unit is assumed to be 2.0 for the calculations.

Sample test cases :

Input 1 :
1
Espresso
150.0
10

Output 1 :
Espresso 1200.00

Input 2 :
2
Latte
200.0
5
Cappuccino
180.0
2

Output 2 :
Latte 900.00
Cappuccino 360.00

PAT4:-

Question No: 1

Single File Programming Question
Problem Statement

Sid needs to calculate property tax for different types of properties: Residential, Commercial, and Industrial. Each type has its own tax rules:

Residential: Tax is the base rate multiplied by a location factor. A 10% discount applies if the property is older than 20 years; otherwise, there's a 5% penalty.

Commercial: Tax is the base rate multiplied by a location factor. A 15% discount applies if the property is older than 15 years; otherwise, there's a 7% penalty.

Industrial: Tax is the base rate multiplied by a location factor. A 20% discount applies if the property is older than 10 years; otherwise, there's a 10% penalty.

Create a base class Property with subclasses Residential, Commercial, and Industrial. Each subclass will override the calculateTax(double baseRate, double locationFactor, int age) method to compute the tax based on its specific rules.

Input format :

The first line of input consists of an integer N, representing the property type (1 for Residential, 2 for Commercial, 3 for Industrial).

The second line of input consists of an integer B, representing the base rate for the property.

The third line of input consists of an integer L, representing the location factor.

The fourth line of input consists of an integer A, representing the age of the property.

Output format :

The output displays an double value representing the calculated property tax rounded off to two decimal places.

Refer to the sample output for the formatting specifications.

Code constraints :

N is either 1, 2 or 3

$500 \leq B \leq 2000$

$1 \leq L \leq 10$

$1 \leq A \leq 30$

Sample test cases :

Input 1 :

1

1500

4

3

Output 1 :

6300.00

Input 2 :

2

800

1

11

Output 2 :

856.00

Input 3 :

3

540

2

4

Output 3 :

1188.00

Question No: 2

Single File Programming Question

Problem Statement

David is planning to invest in various types of properties and wants to estimate the yearly property tax to manage his budget.

Create an interface named Property with a method to calculate the yearly tax.

Implement this interface in classes Residential, Commercial, and Industrial, where each class calculates the tax based on the following criteria:

Residential: Tax = base rate * location factor. If the property age is over 20 years, apply a 10% discount. If the age is 20 years or less, add a 5% penalty.

Commercial: Tax = base rate * location factor. If the property age is over 15 years, apply a 15% discount. If the age is 15 years or less, add a 7% penalty.

Industrial: Tax = base rate * location factor. If the property age is over 10 years, apply a 20% discount. If the age is 10 years or less, add a 10% penalty.

David needs to input the property type, base rate, location factor, and age to determine the yearly tax for each property.

Input format :

The first line of input consists of an integer, representing the property type (1 for Residential, 2 for Commercial, 3 for Industrial).

The second line consists of an integer B, representing the base rate for the

property.

The third line consists of an integer L, representing the location factor.

The fourth line consists of an integer A, representing the age of the property.

Output format :

The output prints a double value representing the calculated property tax, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Code constraints :

$500 \leq B \leq 2000$

$1 \leq L \leq 20$

$1 \leq A \leq 50$

Sample test cases :

Input 1 :

1
1500
4
3

Output 1 :

6300.00

Input 2 :

1
1500
4
24

Output 2 :

5400.00

Input 3 :

2
800
1
11

Output 3 :

856.00

Input 4 :

2
800
1
26

Output 4 :

680.00

Input 5 :

3
540
2
10

Output 5 :

1188.00

Input 6 :

3
540
2
35

Output 6 :

864.00

PAT5: -

Question No: 1

Single File Programming Question Problem Statement

Find the common, and keep the order!

Raj loves working with letters and aims to develop a program that identifies common characters between two given strings, preserving the sequence of the first string. He plans to use Scanner for user input and StringBuffer for efficient character manipulation. The program will display the common characters or a message if no matches are found.

Input format :

The input consists of two strings in separate lines.

Output format :

The output prints the common characters from the first string in the order they appear.

If there are no common characters, print "No common characters".

Refer to the sample output for formatting specifications.

Code constraints :

The maximum length of the input string is 100.

Case-sensitive characters (e.g., 'A' ≠ 'a').

Characters are printed once, even if repeated in the first string.

Sample test cases :

Input 1 :

apple

piano

Output 1 :

a p

Input 2 :

Flower

Power

Output 2 :

o e r

Input 3 :

world

peace

Output 3 :

No common characters

Question No: 2

Single File Programming Question Problem Statement

James wanted to buy some restricted items from the store, but he needed to meet certain requirements. First, he had to be at least 21 years old; if not, he would encounter an `AgeRestrictionException` stating he must be 21 or older to purchase these items. Additionally, there was a limit of 5 items per purchase, and exceeding this would trigger a `PurchaseLimitExceededException`. James also had to keep the total price of his items within a limit of 200; if he went over this, a `PriceLimitExceededException` would be thrown.

After ensuring he met these criteria, James also wanted to evaluate the percentage increase between his highest and lowest bids ($(\text{highestBid} - \text{lowestBid}) / \text{lowestBid} * 100$). He could do this through a method that calculates the percentage increase based on the bid amounts. If all conditions were satisfied, he could confidently proceed with his purchase while

understanding his financial decisions.

Input format :

The first line contains an integer representing the customer's age.

The second line contains a string representing the customer's date of birth (not used for validation in this implementation).

The third line contains an integer representing the total number of items being purchased.

The fourth line contains a double value representing the total price of the purchase.

The fifth line contains a double value representing the highest bid.

The sixth line contains a double value representing the lowest bid.

Output format :

If the validation is successful: Display the percentage increase in the two decimal format:

Difference = <difference>

Percentage Increase = <percentageIncrease>%

If there is an error during validation, output the appropriate error message:

For age restriction: "You must be 21 or older to purchase restricted items"

For purchase limit: "Cannot purchase more than 5 items"

For price limit: "Total price cannot exceed 200"

Refer to the sample output for formatting specifications.

Sample test cases :

Input 1 :

22

1999-06-15

3

150.0

500.0

400.0

Output 1 :

Difference = 100.00

Percentage Increase = 25.00%

Input 2 :

19

2003-09-23

2

100.0

600.0

500.0

Output 2 :

You must be 21 or older to purchase restricted items

Input 3 :

25

1998-01-05

6

180.0

700.0

350.0

Output 3 :

Cannot purchase more than 5 items

Input 4 :

24

1998-12-25
4
210.0
450.0
375.0
Output 4 :
Total price cannot exceed 200

PAT6:-

Single File Programming Question Problem Statement

Ethan, a stock market analyst, wants to determine the optimal time to buy and sell a stock for maximum profit. He uses an ArrayList to store daily stock prices and implements a function to calculate the maximum possible profit. The program takes the number of days and stock prices as input, then identifies two prices with the maximum difference, ensuring the higher price occurs after the lower price, and displays the maximum profit.

Input format :

The first line of input is an integer N, representing the number of days.

The second line contains N space-separated integers, representing the stock price for each day.

Output format :

The output prints a single integer representing the maximum profit that can be made. If no profit can be made, the output will be 0.

Refer to the sample output for formatting specifications.

Code constraints :

The given testcases fall under the following constraints:

$2 \leq N \leq 15$

$1 \leq \text{stock price} \leq 1000$

Sample test cases :

Input 1 :

6

107 101 105 203 206 104

Output 1 :

105

Input 2 :

5

100 101 102 103 104

Output 2 :

4

Input 3 :

5

160 145 130 129 110

Output 3 :

0

Single File Programming Question Problem Statement

Amelia is a linguistics student who is studying sentence structures. She wants to extract sentences that begin with a vowel from a given set of lines for further analysis.

Create a program that reads a number of lines from the user, writes these lines to a file named "input.txt", and then processes this file to extract the lines that start with a vowel (A, E, I, O, U, both uppercase and lowercase). The extracted lines should be saved in a new file named "output.txt".

Implement a class Solution with the required method `extractLinesStartingWithVowel()` to handle the file operations and extraction. Help Amelia with her linguistic analysis by building this vowel line extractor.

Input format :

The first line of input consists of an integer n , representing the number of lines to be input.

The next n lines consist of strings (sentences or words), where each line represents a separate input.

Output format :

The output consists of lines that start with a vowel (either uppercase or lowercase), printed as they appear in the input, in the same order.

Refer to the sample output for formatting specifications.

Code constraints :

$1 \leq n \leq 100$

Sample test cases :

Input 1 :

3

An apple a day keeps the doctor away.

Every cloud has a silver lining.

Some days are better than others.

Output 1 :

An apple a day keeps the doctor away.

Every cloud has a silver lining.

Input 2 :

2

Incredible things happen every day.

This is a test case.

Output 2 :

Incredible things happen every day.